

桀骜的灵魂

【原】AS3抽象工厂、反射工厂 实例讲解

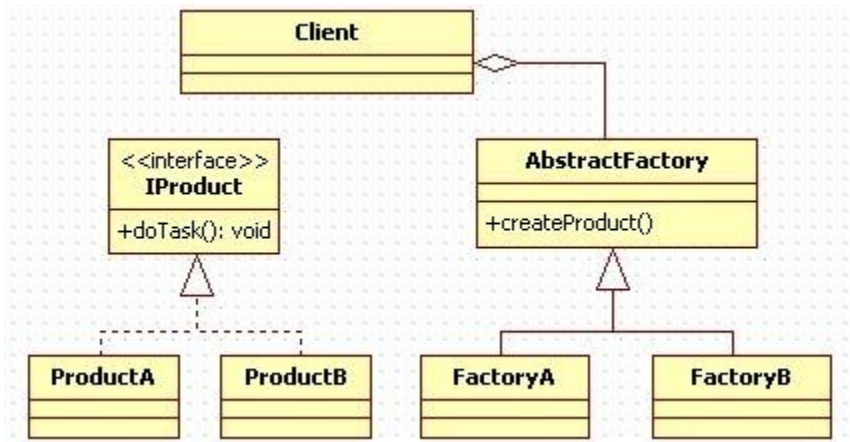
2010-05-22 09:52 by bugfly, 317 阅读, 0 评论, 收藏, 编辑

时间轴动画剪辑器输出编译基错误

反射工厂例子：
产品A生产成功！
产品B生产成功！

本文语言使用的是AS3脚本，通过设计-编码这一标准开发流程阐述，拿出抽象工厂和反射工厂两个实现过程例子，用精简的信息去表达意图。抽象工厂的局限性在于，它没法消除new这些碎片耦合部分，而new碎片散落在客户端上，堆积到一定程度就出现坏味道，甚至出现维护问题，不容易修改，而通过反射机制可以缓解这种new碎片散落程度。如果要优雅地解耦，必然要使用框架，据我所知AS3是有其支持的框架的，由于种种问题，本人没有研究过AS3的框架，不过通过AS3自身的反射机制可以模拟一个解释器去到达最大限度解耦。相信不用我对图片太多解释你也可以明白意思了。而这里涉及到一些UML知识，真有需要请自己查找。

(1) 抽象工厂。



About

最近在读



书名：[ExtJs in Action](#)
作者：jesus
出版日期：2009-5-20
出版社：Manning Publications
页数：255
ISBN：ISBN-00: 978-1847197566

我的信息
QQ:315@09411
欢迎第 38833 位朋友!

推荐博客
[Richard DingWall](#)
[Asuka](#)

```
package{
    //产品接口。
    public interface IProduct{
        function doTask();
    }
}

package{
    //具体产品。
    public class ProductA implements IProduct{
        public function ProductA(){
        }

        public function doTask(){
            trace("产品A生产成功!");
        }
    }
}

package{
    //具体产品。
    public class ProductB implements IProduct{
        public function ProductB(){
        }

        public function doTask(){
            trace("产品B生产成功!");
        }
    }
}
```

```
package{
    //抽象工厂。
    public class AbstractFactory{
        public function AbstractFactory(){
        }

        public function createProduct():IProduct{
            return null;
        }
    }
}

package{
    //具体工厂。
    public class FactoryB extends AbstractFactory{
        public function FactoryB(){
        }

        public override function createProduct():IProduct{
            return new ProductB();
        }
    }
}

package{
    //具体工厂。
    public class FactoryA extends AbstractFactory{
        public function FactoryA(){
        }

        public override function createProduct():IProduct{
            return new ProductA();
        }
    }
}
```

我关注中的CodePlex开源项目
 [Emit Mapper](#)

访问者分布
 

昵称：[bugfly](#)
 园龄：[8年4个月](#)
 荣誉：[推荐博客](#)
 粉丝：[58](#)
 关注：[30](#)
[+加关注](#)

最新评论

日历

<

2010年5月

>

日	一	二	三	四	五	六
25	26	27	28	29	30	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	<u>22</u>
23	24	25	26	27	28	29
30	31	1	2	3	4	5

我的标签

asp.net MVC(5)

IOC(5)

Unity(5)

AOP(4)

DI(3)

ExtJs(3)

MVC(3)

ORM(3)

UML(3)

MVC3(3)

更多

随笔档案

2011年10月(1)

2011年7月(5)

2011年6月(6)

2011年5月(2)

2010年12月(1)

2010年11月(1)

2010年10月(5)

2010年9月(6)

2010年8月(1)

2010年6月(4)

2010年5月(8)

```
package{
import flash.display.*;
//客户端
public class Client extends MovieClip{

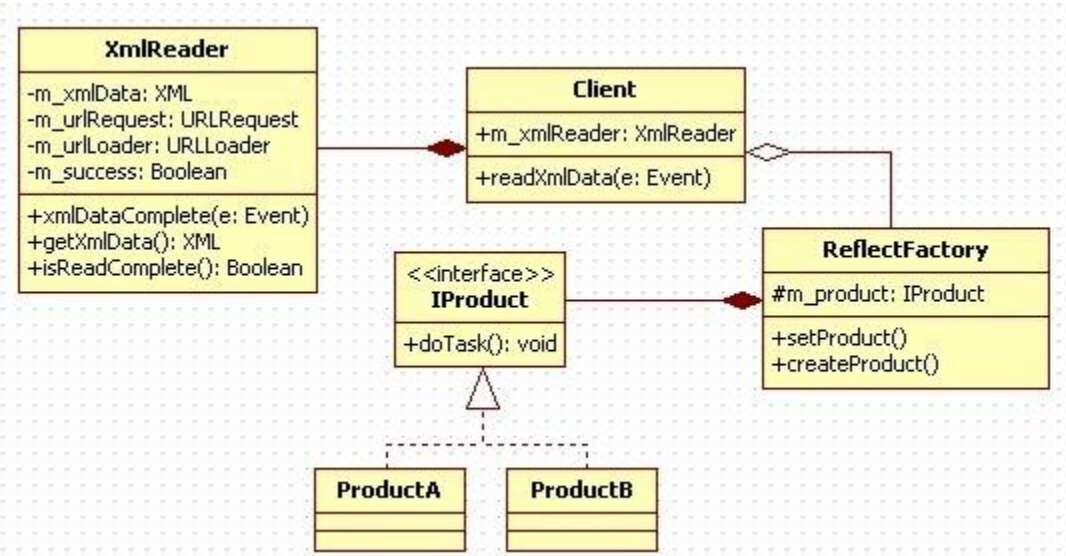
    public function Client() {
        //无论抽象工厂怎么解耦，调用端都要new对应的工厂实例，依然在编码中耦合！
        //FactoryA, FactoryB, 甚至未来的工厂都是一些变化点，也是耦合原因。
        var n_factory:AbstractFactory=new FactoryA();
        n_factory.createProduct().doTask();

        n_factory=new FactoryB();
        n_factory.createProduct().doTask();
    }
}
```

时间轴 动画剪辑器 输出 编译基错误

产品A生产成功!
产品B生产成功!

(2) 反射工厂。



随笔分类

- [.NET框架\(13\)](#)
- [Asp.Net相关\(3\)](#)
- [CQRS\(3\)](#)
- [DDD\(3\)](#)
- [ExtJS\(3\)](#)
- [Flash相关\(2\)](#)
- [Ruby相关\(1\)](#)
- [Spring.Net系列\(3\)](#)
- [程序经验\(5\)](#)
- [胡言乱语\(6\)](#)
- [四色原型\(2\)](#)
- [英语日记\(2\)](#)

推荐排行榜

- [1. Unity 在MVC上的应用\(下:ORM\)\(9\)](#)
- [2. ExtJS & ASP.NET MVC3.0 学习笔记一\(7\)](#)
- [3. 【讨论】“转账功能”引发的“谜团”\(6\)](#)
- [4. 【原】\(一\) Unity 2.0 IOC容器入门教程\(6\)](#)
- [5. Unity 在MVC上的应用\(上:思路整理\)\(6\)](#)

阅读排行榜

- [1. ExtJS & ASP.NET MVC3.0 学习笔记一\(5705\)](#)
- [2. 【原】\(一\) Unity 2.0 IOC容器入门教程\(4767\)](#)
- [3. Unity 在MVC上的应用\(下:ORM\)\(4528\)](#)
- [4. 【讨论】“转账功能”引发的“谜团”\(4156\)](#)
- [5. ExtJS & ASP.NET MVC3.0 学习笔记三\(3787\)](#)

```
package{
    //反射工厂
    public class ReflectFactory{
        private var m_product:IProduct;

        public function ReflectFactory(){
        }

        public function setProduct(n_product:IProduct){
            this.m_product=n_product
        }

        public function createProduct():IProduct{
            return this.m_product;
        }
    }
}

package {
    //XML读取类
    import flash.events.*;
    import flash.utils.*;
    import flash.net.*;

    public class XmlReader{

        private var m_xmlData:XML;
        private var m_urlRequest:URLRequest;
        private var m_urlLoader:URLLoader;
        private var m_success:Boolean;

        public function XmlReader(n_xmlPath:String) {
            this.m_urlRequest=new URLRequest(n_xmlPath);
            this.m_urlLoader=new URLLoader(this.m_urlRequest);
            this.m_success=false;

            this.m_urlLoader.addEventListener(Event.COMPLETE,this.xmlDataComplete);
        }

        public function xmlDataComplete(e:Event) {
            this.m_xmlData=XML(this.m_urlLoader.data);
            this.m_success=true;
        }

        public function getXmlData():XML{
            return this.m_xmlData;
        }

        public function isReadComplete():Boolean{
            return this.m_success;
        }
    }
}
```

```
package{
    import flash.display.*;
    import flash.events.*;
    import flash.utils.*;
    import flash.net.*;
    import flash.system.*;
    //客户端
    public class Client extends MovieClip{

        private var m_xmlReader:XmlReader;
        //预定义产品，因为Flash机制原因要引入。
        //比较正规的反射语言不用这样做的。
        private var m_productA:ProductA;
        private var m_productB:ProductB;

        public function Client() {
            this.m_xmlReader=new XmlReader("test.xml");
            trace("反射工厂例子：");
            addEventListener(Event.ENTER_FRAME,readXmlData);
        }

        public function readXmlData(e:Event){
            var n_factory:ReflectFactory=new ReflectFactory();

            if(this.m_xmlReader.isReadComplete())
            {
                var n_xmlData:XML=this.m_xmlReader.getXmlData();
                for(var i=0;i<n_xmlData.Product.length();i++){
                    var TempClass:Class=getDefinitionByName(n_xmlData.Product[i].@className) as Class;
                    var n_product:IProduct=new TempClass() as IProduct;
                    n_factory.setProduct(n_product);
                    n_factory.createProduct().doTask();
                }
                removeEventListener(Event.ENTER_FRAME,readXmlData);
            }
        }
    }
}
```

配置文件test.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Products>
    <Product className="ProductA" />
    <Product className="ProductB" />
</Products>
```

结语,对于抽象工厂我就不多解释了，反射工厂就有几点补充，可以看出，反射工厂没有在代码区域耦合一点的实现代码，都是通过抽象层面去交互，而也是它取代抽象工厂的原因，而不好的地方是要预先引入产品实体类，这个其实也是不得不做的，根源是AS3的机制问题，而这一点不影响区别两种工厂的优缺点。反射工厂的优势是把变化隔离出外部文件，而消除了具体对象使用的判断逻辑，没有new碎片了，而这是抽象工厂无法做到的，抽象工厂无法摆脱这些判断，而这些判断必然会导致耦合加深，终究会出现new去生成对象。简短例子，希望对你有帮助。

作者：[桀骜的灵魂](#)

出处：<http://www.cnblogs.com/HuntSoul/>

本文版权归作者和博客园共有，欢迎转载，但未经作者同意必须保留此段声明，且在文章页面明显位置给出原文连接，否则保留追究法律责任的权利。

好文要顶

关注我

收藏该文



bugfly

关注 - 30

粉丝 - 58

荣誉：推荐博客

+加关注

« 上一篇：[【原】Flash AS3反射机制简述](#)

» 下一篇：[【原】set get 是否也是接口](#)

0

0

分类: [Flash相关](#)

标签: [Flash](#), [ActionScript3.0](#), [Aa3](#), [抽象工厂](#), [反射工厂](#)

[刷新评论](#) [刷新页面](#) [返回顶部](#)

注册用户登录后才能发表评论，请 [登录](#) 或 [注册](#)，[访问网站首页](#)。

【推荐】超50万VC++源码: 大型组态工控、电力仿真CAD与GIS源码库！

【推荐】企业SaaS应用开发实战，快速构建企业运营/运维系统

【推荐】ActiveReports 报表控件，全面满足 .NET开发需求



腾讯云

如何快速低成本完成网站搭建？

助力 **日PV1~100万** 网站部署，
云产品 **3折** 起

[立即购买](#)

最新IT新闻:

- 苹果股价本月12次创出新高：涨出一个耐克或IBM市值
 - 视频网站13年：土豆、PPTV、酷6网、优酷们的生死书
 - 腾讯开发了一个检查数学作业的小程序
 - 贝索斯每年只从亚马逊领8万工资？美股年报该这么挖
 - 深度解析 | 基于DAG的分布式任务调度平台：Maat
- » [更多新闻...](#)



上海 | 2018.10.10-12

华为全联接大会

[了解详情](#)



最新知识库文章:

- 如何招到一个靠谱的程序员
- 一个故事看懂 “区块链”
- 被踢出去的用户
- 成为一个有目标的学习者
- 历史转折中的 “杭派工程师”
- » 更多知识库文章...